Consistent Partial Least Squares: A cSEM tutorial

Florian Schuberth

September 18, 2022

Florian Schuberth (corresponding author), Faculty of Engineering Technology, University of Twente, P.O. Box 217, 7500 AE Enschede, The Netherlands
E-mail: f.schuberth@utwente.nl, *Phone:* +31 53 4897846

Abstract

This tutorial article provides a comprehensive, step-by-step illustration of consistent partial least squares (PLSc) using the cSEM package in R. PLSc is a variant of partial least squares path modeling, a composite-based approach to structural equation modeling, that produces consistent parameter estimates for latent variable models. In particular, PLSc corrects construct correlations for attenuation using Dijkstra-Henseler's composite reliability coefficient ϱ_A. Building on the employee retention example introduced by **?**, we demonstrate how to install and load cSEM, specify the model using lavaan-style syntax, estimate the PLSc-SEM model, evaluate measurement model quality (internal consistency reliability, convergent validity, and discriminant validity), and assess the structural model (collinearity, effect sizes, model fit, and path coefficients with bootstrap inference). We further contrast the PLSc-SEM results with standard PLS-SEM estimates and discuss practical considerations for using PLSc-SEM in empirical research.

Keywords: measurement error, bias correction, partial least squares structural equation modeling (PLS-SEM), factor score regression

Introduction

Structural equation modeling (SEM) is a versatile multivariate analysis framework used in the behavioral, social, medical, and management sciences to study complex relationships between constructs and their relations with observed variables (e.g., **??**). To estimate structural equation models, various approaches have been proposed, including maximum-likelihood (**?**), weighted least squares (**?**), factor score regression (**??**) and (integrated) generalized structured component analysis (**??**)

A composite-based estimator to SEM that gained popularity over the last decades in various fields of social sciences (**?**), including marketing (**?**), information systems research (**??**), human resource management (**?**), and tourism research (**?**), is partial least squares path modeling (PLS-PM, **?**),also known as partial least squares structural equation modeling (PLS-SEM, **?**). PLS-PM is a two-step estimation procedure (**?**). In the first step, construct scores are obtained by the partial least squares algorithm. Subsequently, these construct scores are used to estimate the model parameters. Although PLS-PM is computationally efficient, it is known that in its original form to produce inconsistent estimates for models involving latent variables due to attenuation bias (e.g., **????**).

To address this limitation, consistent partial least squares (PLSc, **????**) was introduced. PLSc is based on PLS-PM, but applies a correction for attenuation to obtain consistent estimates of reflective measurement models and path coefficient between latent variables. Additionally, PLSc has been extended to deal with ordinal variables (**?**), randomly occurring outliers (**?**) and multicollinearity issues (**?**).

To apply PLS-PM and PLSc, various implementations have been developed (**?**). They can be distinguished into: (i) proprietary software and (ii) open-source software. Proprietary software for conducting PLS-PM and PLSc includes SmartPLS (**?**), WarpPLS (**?**) and ADANCO (**?**). On the other hand, open-source software alternatives include implementations in the statistical programming environment R (**?**) such as matrixpls (**?**), SEMinR (**?**), cSEM (**?**) and plspm (**?**). Similarly, it includes implementations in python (**?**) such as plspm (**?**). While proprietary software such as SmartPLS often shows high ease of use (**?**), they conflict with the open science principles (**?**). For example, their codebase is not openly accessible of transparency (**?**), reproducibility (**?**), and accessibility/inclusiveness (). Open-source software overcomes these limitations (**?**). Therefore, this tutorial presents the R package cSEM as a full-fledged open-source alternative to SmartPLS to apply PLS-PM and PLSc.

?

(**?**)

The remainder of the tutorial is structured as follows. Section Section Section

1 cSEM R package Illustration

1.1 First steps in R

This section illustrates how to apply PLSc using the cSEM package, starting from the very first steps. It assumes no prior experience with R; readers already familiar with R and RStudio may skip directly to Section 1.2.

To use the cSEM package, users first need to install R and an integrated development environment (IDE) such as RStudio. For the installation, we refer to the official guides for R (`https://cran.r-project.org`) and RStudio (`https://posit.co/download/rstudio-desktop`).

When RStudio is opened for the first time, it displays four panes (Figure **??**). The two most important ones are the *script editor* (upper left), where code is written and saved, and the *Console* (lower left), where the code is executed and results are printed. The *Environment*

pane (upper right) lists the objects currently loaded into memory, and the fourth pane (lower right) provides access to files, plots, and help pages.

Although code can be typed directly into the console, it is good practice to work in a script, since a script can be saved, shared, and re-run, which is essential for reproducible analyses. To create a new script, click File → New File → R Script. The typical workflow is to write code in the editor and run the selected line(s) with `Ctrl + Enter`. Any line beginning with `#` is treated as a comment: it documents what the code does and is ignored when the code runs.

```
# This is a comment and is not executed.
1 + 1   # Code is executed; the result is printed in the Console.
```

Before cSEM can be used, it must be installed. The cSEM package is available on CRAN, so the most recent stable version can be installed with:

```
install.packages("cSEM")
```

Installation only needs to be done once per machine (rerun this line to update the package later). Moreover, development versions can be installed from GitHub: `https://github.com/FloSchuberth/cSEM`. After installation, the package must be loaded at the start of every new R session before its functions become available:

```
library(cSEM)
```

With cSEM installed and loaded, we can now import the data and specify the model, as described in the following sections.

1.2 Model and Data

The cSEM package (**?**) offers a variety of composite-based approaches to SEM including PLS-PM and PLSc. To illustrate the application of PLSc using cSEM, we use the data and model used in **?** to demonstrate SmartPLS. Similarly, we compare the results to those of the maximum-likelihood estimation as implemented in the R package lavaan (**??**).

Following **?**, our illustration draws on the employee retention model introduced by **?**, Chapter 13. The model explains the effects of organizational commitment (OC) and job satisfaction (JS) on employees' staying intentions (SI), with work environment perceptions (EP) and attitudes toward coworkers (AC) as exogenous antecedent constructs. Five constructs are reflectively measured by a total of 21 indicators. A detailed description of the indicators can be found in **?**. The model is presented in Figure **??**

The central function is csem(), which accepts a model specification in lavaan-style syntax and a dataset, and returns an object containing all estimation results. Key arguments include:

`.data` A data frame or matrix containing the observed indicator variables.

`.model` A model string written in lavaan syntax, with `=~` for measurement relations and `~` for structural (regression) relations.

`.approach_weights` The weighting scheme for computing composite scores. Set to `"PLS-PM"` for PLS-based estimation (the default).

`.disattenuate` Logical. When set to `TRUE`, cSEM applies the PLSc correction for attenuation, yielding consistent estimates under the common factor model. This is the key switch that distinguishes PLSc-SEM from standard PLS-SEM.

`.PLS_weight_scheme_inner` The inner weighting scheme; options include `"path"` (default), `"centroid"`, and `"factorial"`.

When loading own datasets in R users need to ensure they are formatted as dataframes or matrices before using cSEM functions. To do that users can formate the datasets with as.data.frame() or as.matrix().

Additional post estimation functions include assess() for measurement and structural model quality criteria, summarize() for a comprehensive results summary, infer() for bootstrap-based inference, testOMF() for the overall model fit test, and predict() for out-of-sample prediction.

1.3 Model specification

1.4 PLSc-SEM Estimation

The cSEM R package

The cSEM package (**?**) offers a variety of composite-based approaches to SEM including PLS-PM and PLSc. It has been developed in the statistical programming environment R (**?**) and is available on CRAN under a GNU general public license (GPL). Thus, the most recent stable version can be installed as follows:

```
install.packages("cSEM")
```

Moreover, development versions can be installed from GitHub: `https://github.com/FloSchuberth/cSEM`.

To illustrate the application of PLSc using cSEM, we use the data and model used in **?** to demonstrate SmartPLS. Similarly, we compare the results to those of the maximum-likelihood estimatro as implemented in the R package lavaan (**??**).

The central function is csem(), which accepts a model specification in lavaan-style syntax and a dataset, and returns an object containing all estimation results. Key arguments include:

`.data` A data frame or matrix containing the observed indicator variables.

`.model` A model string written in lavaan syntax, with `=~` for measurement relations and `~` for structural (regression) relations.

`.approach_weights` The weighting scheme for computing composite scores. Set to `"PLS-PM"` for PLS-based estimation (the default).

`.disattenuate` Logical. When set to `TRUE`, cSEM applies the PLSc correction for attenuation, yielding consistent estimates under the common factor model. This is the key switch that distinguishes PLSc-SEM from standard PLS-SEM.

`.PLS_weight_scheme_inner` The inner weighting scheme; options include `"path"` (default), `"centroid"`, and `"factorial"`.

Additional postestimation functions include assess() for measurement and structural model quality criteria, summarize() for a comprehensive results summary, infer() for bootstrap-based inference, testOMF() for the overall model fit test, and predict() for out-of-sample prediction.

1.5 Model and Data

Following **?**, our illustration draws on the employee retention model introduced by **?**, Chapter 13. The model explains the effects of organizational commitment (OC) and job satisfaction (JS) on employees' staying intentions (SI), with work environment perceptions (EP) and

attitudes toward coworkers (AC) as exogenous antecedent constructs. Five constructs are reflectively measured by a total of 21 indicators. A detailed description of the indicators can be found in **?**. The model is presented in Figure **??**

The dataset used for model estimation is publicly available from the SmartPLS website: `https://www.smartpls.com/documentation/sample-projects/employee-retention` The synthetic dataset comprises $N = 400$ observations from HBAT Industries and was generated to satisfy the assumptions of factor-based SEM, including multivariate normality. For this tutorial, we assume the data have been downloaded as a CSV file. In the following, the dataset is imported to R and the relevant variables are selected.

```
# Import dataset
data <- read.csv("HBAT_SEM.csv", sep=";")

# Select relevant variables
data_rel <- data[, c("AC1", "AC2", "AC3", "AC4",
                     "EP1", "EP2", "EP3", "EP4",
                     "JS1", "JS2", "JS3", "JS4", "JS5",
                     "OC1", "OC2", "OC3", "OC4",
                     "SI1", "SI2", "SI3", "SI4")]

dim(data_rel)
```

1.6 Model Specification

In cSEM, models are specified using lavaan model syntax. In particular, the `=~` and the `<~` operators are used to define the relations between the constructs and their indicators. Specifically, the `=~` operator is used to specify reflective measurement models, while the `<~` is used to specify composite models (see , for an explanation of reflective and composite models). In addition, the operator is used to specify the relations among constructs. In case, an observed variable should be directly accounted for in the structural model, it must be specified as a single indicator construct, as common in PLS-PM (). The cSEM specification of the retention model illustrated in Figure **??** is as follows:

```
model <- "
  # Measurement model
  AC =~ AC1 + AC2 + AC3 + AC4
  EP =~ EP1 + EP2 + EP3 + EP4
  JS =~ JS1 + JS2 + JS3 + JS4 + JS5
  OC =~ OC1 + OC2 + OC3 + OC4
  SI =~ SI1 + SI2 + SI3 + SI4

  # Structural model
  JS ~ EP + AC
  OC ~ EP + AC + JS
  SI ~ JS + OC
"
```

1.7 PLSc-SEM Estimation

The workhorse of the cSEM package is the csem function. The minimal input required for the csem fucntion is a dataset and a model. In addition, the csem function shows various arguments

that can be used to adjust the PLS algorithm. These arguments include

To ultimately estimate the model using PLSc-SEM, we call csem() with `.disattenuate = TRUE`. This single argument activates the consistent correction that distinguishes PLSc-SEM from standard PLS-SEM:

```
# PLSc-SEM estimation
res_plsc <- csem(
  .data                    = hbat_sub,
  .model                   = model,
  .approach_weights        = "PLS-PM",
  .disattenuate            = TRUE,
  .PLS_weight_scheme_inner = "path"
)

# Verify convergence
verify(res_plsc)
```

The verify() function checks whether the PLS algorithm converged, whether all construct reliability estimates (ρ_A) are positive (a necessary condition for the PLSc correction), and whether the model-implied indicator correlation matrix is admissible. If all checks pass, the estimation can be considered successful.

To obtain a comprehensive overview of all results, including loadings, path coefficients, reliability measures, and R^2 values, use:

```
# Full results summary
summary_res <- summarize(res_plsc)
print(summary_res)

# For a more compact overview
summary_res$Estimates$Loading_estimates
summary_res$Estimates$Path_estimates
```

For comparison, standard PLS-SEM (without the consistency correction) can be obtained by setting `.disattenuate = FALSE`:

```
# Standard PLS-SEM (without correction)
res_pls <- csem(
  .data             = hbat_sub,
  .model            = model,
  .approach_weights = "PLS-PM",
  .disattenuate     = FALSE
)
```

1.8 Measurement Model Assessment

Following the guidelines described in **?**, Chapter 4 and **?**, we assess the reflective measurement model in terms of internal consistency reliability, convergent validity, and discriminant validity. The assess() function in cSEM provides all relevant quality criteria:

```
# Compute all quality criteria
quality <- assess(res_plsc)

# Internal consistency reliability
quality$Cronbachs_alpha       # Cronbach's alpha
```

```
quality$RhoA                    # Dijkstra-Henseler rho_A
quality$RhoC                    # Composite reliability (Joreskog rho_C)

# Convergent validity
quality$AVE                     # Average variance extracted

# Indicator loadings
summary_res$Estimates$Loading_estimates
```

Table 1: Reflective measurement model assessment results (PLSc-SEM via cSEM).

Construct	Item	Outer loading	Cronbach's α	ρ_A	ρ_C	AVE
AC	AC1	0.770	0.894	0.911	0.926	0.679
	AC2	0.677				
	AC3	0.823				
	AC4	0.993				
EP	EP1	0.631	0.857	0.869	0.903	0.605
	EP2	0.875				
	EP3	0.763				
	EP4	0.821				
JS	JS1	0.618	0.844	0.856	0.888	0.518
	JS2	0.696				
	JS3	0.715				
	JS4	0.628				
	JS5	0.903				
OC	OC1	0.427	0.832	0.887	0.886	0.573
	OC2	0.958				
	OC3	0.682				
	OC4	0.852				
SI	SI1	0.817	0.889	0.899	0.923	0.672
	SI2	0.870				
	SI3	0.677				
	SI4	0.897				

Note. AC = attitudes toward coworkers; EP = environment perceptions; JS = job satisfaction; OC = organizational commitment; SI = staying intentions. Results replicate those in **?**, Table 1.

The results in Table 1 indicate that all constructs meet the commonly accepted thresholds. All Cronbach's α and ρ_A values exceed 0.70, and all AVE values exceed 0.50, supporting internal consistency reliability and convergent validity. While some outer loadings (e.g., OC1 = 0.427) fall below the recommended threshold of 0.708, these indicators are retained because the corresponding constructs exhibit adequate reliability and AVE, and their retention preserves content validity (**?**).

1.8.1 Discriminant Validity: The HTMT Criterion

Discriminant validity is assessed using the heterotrait-monotrait ratio of correlations (HTMT; **?**). In cSEM, HTMT values are available via assess():

```
# HTMT values
quality$HTMT
```

```
# Bootstrap confidence intervals for HTMT
res_plsc_boot <- csem(
  .data                = hbat_sub,
  .model               = model,
  .approach_weights    = "PLS-PM",
  .disattenuate        = TRUE,
  .resample_method     = "bootstrap",
  .R                   = 10000,
  .seed                = 42
)

# Extract HTMT inference
htmt_infer <- infer(res_plsc_boot)
# The infer() output includes CIs for all quality criteria
# including HTMT values
```

Table 2: HTMT criterion results.

Construct	AC	EP	JS	OC
EP	0.257 [0.163; 0.347]			
JS	0.066 [0.032; 0.075]	0.244 [0.164; 0.326]		
OC	0.275 [0.195; 0.355]	0.495 [0.405; 0.580]	0.209 [0.118; 0.297]	
SI	0.310 [0.222; 0.395]	0.569 [0.471; 0.656]	0.232 [0.154; 0.313]	0.501 [0.410; 0.586]

Note. Values in brackets: 90% percentile bootstrap confidence intervals (10,000 subsamples).

The HTMT results in Table 2 confirm discriminant validity: none of the upper bounds of the 90% bootstrap confidence intervals exceed the conservative threshold of 0.85 (**??**). These findings are identical to those reported by **?**, Table 2, confirming that the cSEM results replicate the SmartPLS output.

1.9 Structural Model Assessment

1.9.1 Collinearity Assessment

Before interpreting the structural model, we check for collinearity among predictor constructs by examining variance inflation factors (VIF). In cSEM, VIF values are available through the assess() function:

```
# VIF values for structural model
quality$VIF
```

All VIF values should be below the critical threshold of three (**?**), indicating that multicollinearity is not a concern.

1.9.2 Path Coefficients and Bootstrap Inference

To evaluate the statistical significance of the structural relationships, we rely on bootstrap confidence intervals. The bootstrap estimation was already conducted above. Path coefficients and their confidence intervals are extracted as follows:

```
# Extract path coefficient estimates
summarize(res_plsc)$Estimates$Path_estimates
```

```
# Bootstrap inference for path coefficients
boot_infer <- infer(res_plsc_boot)

# Percentile bootstrap confidence intervals (95%)
boot_infer$Path_estimates$CI_percentile
```

Table 3: Structural model assessment results.

Relationship	Path coeff.	SD	t value	p value	95% CI	VIF	f^2
AC → JS	−0.002	0.058	0.043	.966	[−0.119; 0.106]	1.055	0.000
AC → OC	0.172	0.048	3.619	.000	[0.078; 0.264]	1.055	0.039
EP → JS	0.245	0.053	4.641	.000	[0.136; 0.343]	1.055	0.060
EP → OC	0.430	0.059	7.272	.000	[0.309; 0.540]	1.101	0.227
JS → OC	0.095	0.052	1.820	.069	[−0.011; 0.194]	1.047	0.012
JS → SI	0.132	0.046	2.833	.005	[0.043; 0.225]	1.035	0.023
OC → SI	0.490	0.051	9.520	.000	[0.386; 0.587]	1.035	0.321

Note. CI = 95% percentile bootstrap confidence interval (10,000 subsamples). Results replicate **?**, Table 3.

The structural model results in Table 3 are consistent with those reported by **?**, Table 3. Based on the 95% percentile bootstrap confidence intervals, all structural relationships are statistically significant except for the paths from AC to JS ($\beta = -0.002$, $p = .966$) and from JS to OC ($\beta = 0.095$, $p = .069$). Environment perceptions (EP) exert the strongest positive effect on organizational commitment ($\beta = 0.430$), which in turn is the dominant predictor of staying intentions ($\beta = 0.490$).

1.9.3 Effect Sizes

Cohen's f^2 effect sizes quantify the substantive impact of each predictor. In cSEM:

```
# Effect sizes
quality$F2
```

The paths EP → OC ($f^2 = 0.227$) and OC → SI ($f^2 = 0.321$) show medium to large effects, while the remaining significant paths exhibit small effects. The AC → JS relationship has a trivial effect size of essentially zero.

1.9.4 Model Fit Assessment

Following **?**, we assess model fit using the testOMF() function, which implements a bootstrap-based test comparing the empirical and model-implied correlation matrices:

```
# Overall model fit test
fit_test <- testOMF(
  res_plsc,
  .R                    = 10000,
  .seed                 = 42,
  .handle_inadmissibles = "replace"
)

# Print results
fit_test
```

```
# The output includes:
# - SRMR and its bootstrap-based critical value
# - dULS (squared Euclidean distance) and critical value
# - dG (geodesic distance) and critical value
```

Table 4: Model fit statistics for PLSc-SEM estimation.

	Saturated model	Estimated model
SRMR	0.044	0.071
d_{ULS}	0.438; $\text{UB}_{95\%}$: 0.636	1.155; $\text{UB}_{95\%}$: 0.641
d_{G}	0.303; $\text{UB}_{95\%}$: 0.953	0.330; $\text{UB}_{95\%}$: 0.870

Note. $\text{UB}_{95\%}$ = 95th percentile of the bootstrap distribution (10,000 subsamples).

Consistent with **?**, Table 4, the SRMR of 0.071 falls below the conventional 0.08 threshold (**?**), supporting adequate model fit. The bootstrap-based test yields mixed results: d_{ULS} exceeds its critical value (rejecting fit), while d_{G} falls below its critical value (supporting fit). Overall, the model exhibits a satisfactory level of fit.

2 Comparison with CB-SEM via lavaan

A key advantage of working in R is the seamless ability to compare PLSc-SEM results with those from CB-SEM using the lavaan package (**?**). This allows researchers to conduct the multimethod estimation strategy advocated by **?** and **?** entirely within one script:

```
# CB-SEM via lavaan
library(lavaan)

lavaan_model <- "
  # Measurement model
  AC =~ AC1 + AC2 + AC3 + AC4
  EP =~ EP1 + EP2 + EP3 + EP4
  JS =~ JS1 + JS2 + JS3 + JS4 + JS5
  OC =~ OC1 + OC2 + OC3 + OC4
  SI =~ SI1 + SI2 + SI3 + SI4

  # Structural model
  JS ~ EP + AC
  OC ~ EP + AC + JS
  SI ~ JS + OC
"

fit_cbsem <- sem(lavaan_model, data = hbat_sub)
summary(fit_cbsem, fit.measures = TRUE, standardized = TRUE)

# Extract standardized path coefficients
parameterEstimates(fit_cbsem, standardized = TRUE) |>
  subset(op == "~", select = c(lhs, op, rhs, est, std.all, pvalue))
```

The lavaan model specification is identical to the cSEM specification, making side-by-side comparison straightforward. We can extract and compare the structural path coefficients:

```
# Compare structural path coefficients
# PLSc-SEM
plsc_paths <- summarize(res_plsc)$Estimates$Path_estimates

# CB-SEM
cbsem_paths <- parameterEstimates(fit_cbsem, standardized = TRUE)
cbsem_struct <- cbsem_paths[cbsem_paths$op == "~",
                            c("lhs", "rhs", "std.all")]

# Model fit comparison
fitMeasures(fit_cbsem, c("chisq", "df", "pvalue", "rmsea",
                         "rmsea.ci.lower", "rmsea.ci.upper",
                         "srmr", "cfi", "tli", "nfi"))
```

In line with the findings of **?**, Figure 4, the CB-SEM and PLSc-SEM path coefficient estimates closely correspond. The CB-SEM model fit indices (RMSEA = 0.038, SRMR = 0.060, CFI = 0.976) confirm good model fit, supporting the conclusions drawn from the PLSc-SEM analysis.

3 Comparing PLSc-SEM with Standard PLS-SEM

An instructive exercise is to compare PLSc-SEM with standard PLS-SEM to see the effect of the consistency correction. In cSEM, this is achieved simply by toggling `.disattenuate`:

```
# Standard PLS-SEM (no correction)
res_pls <- csem(
  .data               = hbat_sub,
  .model              = model,
  .approach_weights   = "PLS-PM",
  .disattenuate       = FALSE
)

# Side-by-side comparison
plsc_est <- summarize(res_plsc)$Estimates$Path_estimates
pls_est  <- summarize(res_pls)$Estimates$Path_estimates

comparison <- merge(
  plsc_est[, c("Name", "Estimate")],
  pls_est[, c("Name", "Estimate")],
  by = "Name", suffixes = c("_PLSc", "_PLS")
)
print(comparison)
```

As expected from the methodological literature (**?**), the standard PLS-SEM path coefficients are attenuated (biased toward zero) relative to the PLSc-SEM estimates. The attenuation is most visible for paths involving constructs with lower reliability (e.g., OC, which has a relatively low loading for OC1). The PLSc correction recovers the consistent estimates that would be expected under the common factor model.

4 A Compact Reproducible Workflow

For convenience, we present a compact, self-contained R script that reproduces the complete PLSc-SEM analysis. This script can serve as a template for researchers applying PLSc-SEM with cSEM to their own datasets:

```
# ============================================================
# PLSc-SEM Analysis with cSEM: Compact Reproducible Script
# ============================================================

# 1. Setup ----
library(cSEM)

hbat <- read.csv("HBAT_Employee_Retention.csv", sep = ";")
vars <- c(paste0("AC", 1:4), paste0("EP", 1:4),
          paste0("JS", 1:5), paste0("OC", 1:4),
          paste0("SI", 1:4))
dat <- hbat[, vars]

# 2. Model specification ----
model <- "
  AC =~ AC1 + AC2 + AC3 + AC4
  EP =~ EP1 + EP2 + EP3 + EP4
  JS =~ JS1 + JS2 + JS3 + JS4 + JS5
  OC =~ OC1 + OC2 + OC3 + OC4
  SI =~ SI1 + SI2 + SI3 + SI4
  JS ~ EP + AC
  OC ~ EP + AC + JS
  SI ~ JS + OC
"

# 3. PLSc-SEM estimation ----
res <- csem(dat, model,
            .approach_weights = "PLS-PM",
            .disattenuate = TRUE)
verify(res)

# 4. Assessment ----
q <- assess(res)
q$Cronbachs_alpha; q$RhoA; q$RhoC; q$AVE
q$HTMT; q$VIF; q$F2

# 5. Bootstrap inference ----
res_boot <- csem(dat, model,
                 .approach_weights = "PLS-PM",
                 .disattenuate = TRUE,
                 .resample_method = "bootstrap",
                 .R = 10000, .seed = 42)
infer(res_boot)

# 6. Model fit ----
```

```
testOMF(res, .R = 10000, .seed = 42,
        .handle_inadmissibles = "replace")
```

5 Concluding Remarks

This tutorial has demonstrated how to conduct a complete PLSc-SEM analysis using the cSEM package in R, replicating the employee retention model analysis presented by **?** using SmartPLS 4. The results obtained via cSEM are identical to those reported in the SmartPLS-based tutorial, confirming that both software implementations produce consistent estimates.

The cSEM package offers several advantages for researchers seeking a script-based, open-source approach to PLSc-SEM. First, the use of R scripts ensures full reproducibility: every step from data import to final inference is documented in executable code that can be shared, reviewed, and re-run. Second, working within the R ecosystem provides seamless integration with CB-SEM via lavaan, enabling the multimethod estimation strategy advocated by **?** and **?** within a single analytical pipeline. Third, cSEM provides a rich set of postestimation tools—including testOMF() for model fit, predict() for out-of-sample prediction, and testMICOM() for measurement invariance—that support a comprehensive model evaluation workflow.

Several practical considerations warrant mention. Like SmartPLS, cSEM allows toggling between standard PLS-SEM and PLSc-SEM with a single argument (`.disattenuate`), making it straightforward to assess the impact of the consistency correction. Furthermore, cSEM supports additional weighting approaches beyond PLS-PM, including generalized structured component analysis (GSCA) weights and various regression-based approaches, offering flexibility for sensitivity analyses.

However, users should be aware of certain limitations. The PLSc correction requires that all ρ_A estimates are positive and that the resulting corrected correlation matrix is positive definite. The verify() function checks these conditions, but researchers should inspect warning messages carefully, especially with small samples or weakly measured constructs. As **?** note, the attenuation correction in PLSc-SEM alters the original PLS-SEM path coefficients, which means that predictive assessment methods such as $\text{PLS}_{\text{predict}}$ (**?**) are not directly applicable in the PLSc framework.

We hope this tutorial serves as a practical guide for researchers who wish to leverage the strengths of PLSc-SEM within an open-source, reproducible analytical framework. Coupled with the SmartPLS-based tutorial by **?**, researchers now have comprehensive guidance for conducting PLSc-SEM analyses across both GUI-based and script-based software environments.